

Programmation en C

John Samuel

CPE Lyon

Année: 2025-2026

Courriel: john.samuel@cpe.fr



THE C PROGRAMMING LANGUAGE

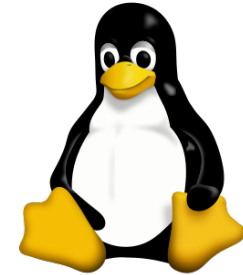
Questions

Donnez des exemples de logiciels écrits en C

1.2. Programmation en C: Histoire

Exemples

1. Noyau Linux: <https://github.com/torvalds/linux>
2. GCC: <https://gcc.gnu.org/git/gitweb.cgi?p=gcc.git>
3. GIT: <https://git.kernel.org/pub/scm/git/git.git/>
4. Apache HTTP Server: <https://github.com/apache/httpd>
5. Python: <https://github.com/python/cpython>



Le langage C est un langage de programmation :

- **Impératif**, où les instructions sont données dans un ordre séquentiel pour définir le comportement du programme.
- **Procédural** car il se concentre sur la création de fonctions et de procédures pour organiser et structurer le code.
- **Structuré** car le C favorise la structuration du code en utilisant des structures de contrôle telles que les boucles et les instructions conditionnelles pour améliorer la lisibilité et la maintenabilité du code.
- **Compilé** car les programmes en C sont compilés, c'est-à-dire qu'ils sont traduits en code machine spécifique à la plateforme avant leur exécution, ce qui permet une exécution rapide et efficace.

1.2. Programmation en C: Histoire

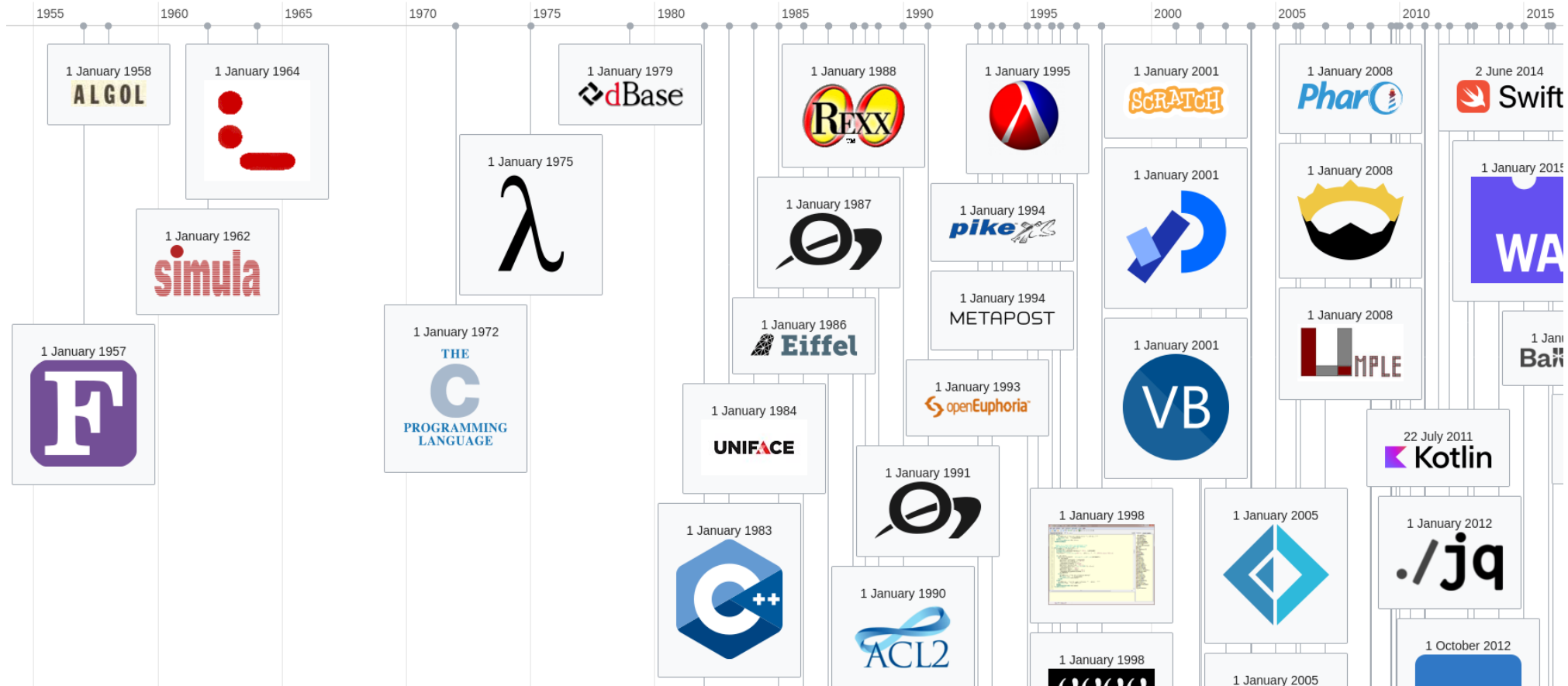
- **Création de C (1972)** par Dennis Ritchie chez Bell Labs
- **Publication du livre "The C Programming Language" (1978)** par Brian Kernighan et Dennis Ritchie
- **Normalisation ANSI C (1989)**, également connue sous le nom de C89. Cela a établi un ensemble de règles strictes pour le langage C, rendant le code plus portable entre les différents systèmes.
- **Introduction de C99 (1999)** qui a apporté de nombreuses améliorations au langage, notamment de nouvelles fonctionnalités telles que les commentaires de style C++, les types de données booléens
- **Développement de C11 (2011)** qui a introduit des fonctionnalités telles que les threads, les types de données complexes, et les alignements
- **Utilisation dans les systèmes d'exploitation** comme Unix, Linux pour le développement
- **Influence sur d'autres langages** de programmation, notamment C++, C#, Objective-C, et plus récemment Rust.
- **Développement d'outils et de bibliothèques**
- **Adoption dans l'embarqué et l'IoT** (Internet des objets)

1.2. Programmation en C: Histoire

Caractéristique	Langage C	Python
Paradigme de programmation	Impératif, procédural	Multiparadigme (impératif, orienté objet, fonctionnel)
Typage	Statique (typage fort)	Dynamique (typage fort) / Statique (Python 3.5+)
Syntaxe	Syntaxe stricte et rigide	Syntaxe simple et lisible
Performance	Performant, optimisé pour les performances	Moins performant, interprété
Gestion de la mémoire	Besoin de gérer manuellement la mémoire	Gestion automatique de la mémoire (ramassage des ordures)
Portabilité	Portable, mais nécessite une compilation spécifique pour chaque plateforme	Hautement portable grâce à son interpréteur

1.2. Programmation en C: Histoire

C: chronologie des langages de



1.3. Bonjour le Monde!!

```
/* Fichier: bonjour1.c
 * affiche 'Bonjour le Monde!!!' à l'écran.
 * auteur: John Samuel
 * Ceci est un commentaire sur plusieurs lignes
 */

#include <stdio.h> // headers

// Ceci est un commentaire sur une ligne
int main() {
    printf("Bonjour le Monde !!!");
    return 0;
}
```

Objectif : Ce code utilise des **commentaires**, inclut le fichier d'en-tête `<stdio.h>` pour l'utilisation de `printf`, définit la **fonction main** comme point d'entrée, utilise `printf` pour afficher un message, et renvoie 0 pour indiquer **une exécution sans erreur**.

1.3. Bonjour le Monde!!

```
/* Fichier: bonjour2.c
 * affiche un message à l'écran en utilisant une variable
 * auteur: John Samuel
 * Ceci est un commentaire sur plusieurs lignes
 */

#include <stdio.h> // headers

int main() {
    int annee = 2025; //déclaration d'une variable
    printf("Bonjour le Monde!!! C'est l'annee %d", annee);
    return 0;
}
```

Objectif : Ce code déclare une variable `annee`, utilise `printf` pour afficher un message incluant la valeur.

1.4. Compilation

La compilation

```
$ gcc bonjour1.c
```

L'exécution

```
$ ./a.out
```

```
Bonjour le Monde!!!
```

1.4. La compilation

La compilation

```
$ gcc -o bonjour bonjour2.c
```

L'exécution

```
$ ./bonjour
```

```
Bonjour le Monde!!! C'est l'annee 2025
```

Pour voir tous les **avertissements** (*warnings*) pendant la compilation, vous pouvez utiliser les options *-Wall* et *-Wextra*.

```
$ gcc -Wall -Wextra -o bonjour bonjour.c
```

1.5. Les commentaires

Commentaires sur une ou plusieurs

```
// Ceci est un commentaire sur une ligne (c99)
```

```
/* Ceci est un  
 * commentaire sur  
 * quatre lignes  
 */
```

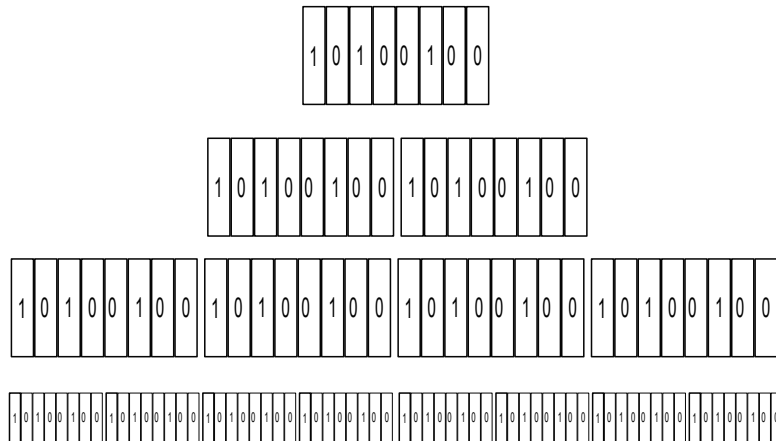
1.6. Les types de base

Les **types de base** en C sont les fondations sur lesquelles sont construites toutes les variables et données dans ce langage de programmation.

Types	Mots clés	Exemples
caractères	char	'h', 'a', ...
entiers	short, int, long, long long	...,-1,0,1,...
nombres en flottant	float, double, long double	3.14, 3.14e23
énumérations	enum	ETUDIANT, STAGIAIRE

1.7. Les types caractères et entiers

Le choix de stocker des données en un octet, deux octets, quatre octets ou huit octets dépend principalement de la précision nécessaire pour représenter les données.



Un octet, deux octets, quatre octets ou huit octets

1.7. Les types caractères et entiers

Types	Mots-clés
caractère	signed char, unsigned char
entier	signed short, signed int, signed long, signed long long, unsigned short, unsigned int, unsigned long, unsigned long long

Remarque : La taille des types de base en C n'est pas standardisée car elle dépend de l'architecture matérielle sous-jacente du système sur lequel le code est exécuté, ce qui permet d'optimiser l'utilisation de la mémoire et les performances pour chaque plate-forme.

1.8. Declaration des variables

Variables char

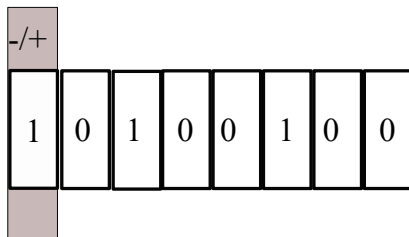
Les variables de type "char" en C sont utilisées pour stocker des caractères, mais elles peuvent également être modifiées avec les qualificateurs "signed" (signé) ou "unsigned" (non signé) pour déterminer si elles peuvent représenter des valeurs négatives (signées) ou uniquement des valeurs positives (non signées).

```
char my_char_var1 = 'a';
```

```
char my_char_var2 = -125;
```

```
unsigned char my_char_var3 = 225;
```

Remarque: Remarquez-bien l'utilisation de sous-tiret en nommant les variables



1.8. Declaration des variables

Variables

```
char my_char_var = 'a';
unsigned char my_uchar_var = 234;
short my_short_var = -12;
unsigned short my_ushort_var = 65535;
int my_int_var = 12;
unsigned int my_uint_var = 3456;
long my_long_var = -1234553L;
unsigned long my_ulong_var = 234556UL;
long long my_llong_var = 1123345LL;
unsigned long long my_ullong_var = 1234567ULL;
float my_float_var = 3.14;
double my_double_var = 3.14E-12;
long double my_long_double_var = 3.14E-22;
```

1.8. Declaration des variables

Énumérations

Les énumérations en C sont un moyen de définir un nouveau type de données personnalisé pour représenter un ensemble fini de valeurs entières nommées. Elles sont utiles pour rendre le code plus lisible et compréhensible.

```
enum status {ETUDIANT, STAGIAIRE}; // ETUDIANT = 0, STAGIAIRE = 1
enum status s = ETUDIANT;
enum status {ETUDIANT=1, STAGIAIRE}; // STAGIAIRE = 2
enum status {ETUDIANT=1, STAGIAIRE=14, ENSEIGNANT}; // ENSEIGNANT = 15
enum boolean {FAUX=0, VRAI};
```

Remarque: `enum`: unsigned int

1.9. L'intervalle minimum et maximum

limits.h

L'intervalle minimum et maximum de types de base en utilisant limits.h

Mots clés	Intervalle
signed char	[SCHAR_MIN, SCHAR_MAX]
unsigned char	[0, UCHAR_MAX]

1.9. L'intervalle minimum et maximum

Mots clés	Intervalle
(signed) short int	[SHRT_MIN, SHRT_MAX]
unsigned short int	[0, USHRT_MAX]
(signed) int	[INT_MIN, INT_MAX]
unsigned int	[0, UINT_MAX]
(signed) long	[LONG_MIN, LONG_MAX]
unsigned long	[0, ULONG_MAX]
(signed) long long	[LLONG_MIN, LLONG_MAX]
unsigned long long	[0, ULLONG_MAX]

1.9. L'intervalle minimum et maximum

float.h

L'intervalle minimum et maximum de types flottant en utilisant float.h

Mots clés	Intervalle
float	[FLT_MIN, FLT_MAX]
double	[DBL_MIN, DBL_MAX]
long double	[LDBL_MIN, LDBL_MAX]

1.10. taille (en octets)

sizeof

```
sizeof (char) //type
sizeof (my_uchar_var) //variable
printf("sizeof(char): %lu\n", sizeof (char));
printf("sizeof(long long int): %lu\n", sizeof (long long int));
printf("sizeof(my_char_var): %lu\n", sizeof (my_char_var));
```

Remarque 1: sizeof donne la taille d'une variable en octets ou d'un type de données.

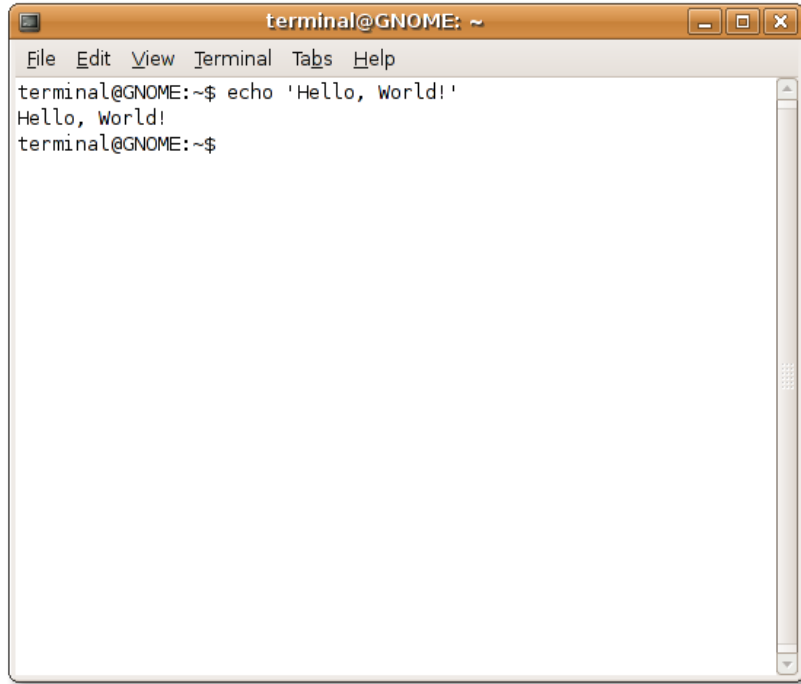
Remarque 2: La valeur retournée peut varier, mais la taille d'une variable de type caractère est de 1 octet.

Remarque 3: La valeur retournée est du type "unsigned long int". C'est pourquoi nous utilisons "%lu".

1.11. Afficher à l'écran

Afficher à l'écran

```
printf("%d", my_int_var);  
printf("%f", my_float_var);
```



1.11. printf: Afficher à l'écran

Mots clés	Code de conversion
char	c
unsigned char	hu
short	hd
unsigned short	hu
int	d, i
unsigned int	u
long int	ld
unsigned long int	lu

1.11. printf: Afficher à l'écran

Mots clés	Code de conversion
long long int	lld
unsigned long long int	llu
float	f, F, e, E
double	g, G
long double	Lg, LG, Le, LE
string of characters	s

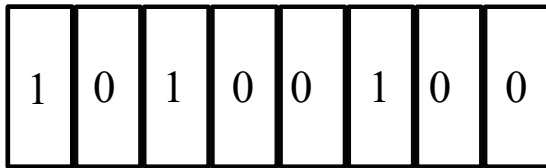
1.11. printf: Afficher à l'écran

Character	Code de conversion
Retour-chariot	<code>\n</code>
Tabulation	<code>\t</code>

1.12. Les notations

Notation binaire

La notation binaire en C permet de représenter des nombres en utilisant uniquement les chiffres 0 et 1, en préfixant les valeurs binaires par "0b", ce qui facilite la manipulation des données au niveau des bits.

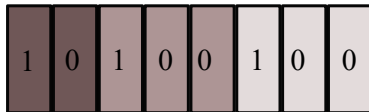


```
int valeur = 0b10100100;
```

1.12. Les notations

Notation octale

La notation octale en C permet de représenter des nombres en utilisant la base octale, c'est-à-dire en utilisant les chiffres de 0 à 7, en préfixant les valeurs octales par "0".



```
int valeurb = 0b10100100;  
int valeuro = 0244;  
printf("notation octale: %o\n", valeurb);  
printf("notation octale: %o\n", valeuro);
```

Notation octale

0	0	0	0
1	0	0	1
2	0	1	0
3	0	1	1
4	1	0	0
5	1	0	1
6	1	1	0
7	1	1	1

1.12. Les notations

Notation hexadécimale

La notation hexadécimale en C permet de représenter des nombres en utilisant la base hexadécimale, c'est-à-dire en utilisant les chiffres de 0 à 9 et les lettres de A à F, en préfixant les valeurs hexadécimales par "0x".

```
int valeurb = 0b10100100;  
int valeurx = 0xa4;  
printf("notation hexadecimale: %x\n",  
       valeurb);  
printf("notation hexadecimale: %x\n",  
       valeurx);
```

1	0	1	0	0	1	0	0
---	---	---	---	---	---	---	---

0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1

8	1	0	0	0
9	1	0	0	1
a	1	0	1	0
b	1	0	1	1
c	1	1	0	0
d	1	1	0	1
e	1	1	1	0
f	1	1	1	1

1.12. Les notations

Notation hexadécimale

0	0	0	0	0
1	0	0	0	1
2	0	0	1	0
3	0	0	1	1
4	0	1	0	0
5	0	1	0	1
6	0	1	1	0
7	0	1	1	1

8	1	0	0	0
9	1	0	0	1
a	1	0	1	0
b	1	0	1	1
c	1	1	0	0
d	1	1	0	1
e	1	1	1	0
f	1	1	1	1

1.13. Les opérateurs arithmétiques

Les opérateurs arithmétiques en C sont des symboles spéciaux tels que "+", "-", "*", "/", et "%" utilisés pour effectuer des opérations mathématiques telles que l'addition, la soustraction, la multiplication, la division et le modulo sur des valeurs numériques, facilitant ainsi la manipulation des données numériques dans les programmes.

Opérateur	Objectif
+	addition
-	soustraction
*	multiplication
/	division
%	modulus

1.13. Les opérateurs arithmétiques

```
int a = 20, b = 10;
```

Opérateur	Exemple	Résultat
+	$a + b$	30
-	$a - b$	10
*	$a * b$	200
/	a / b	2
%	$a \% b$	0

1.14. Les opérateurs relationnels

Les opérateurs relationnels en C, tels que "=", "!", "<", ">", "<=", et ">=", sont utilisés pour comparer des valeurs numériques ou des expressions et renvoient une valeur booléenne (vrai ou faux) en fonction de la relation spécifiée, ce qui permet de prendre des décisions conditionnelles dans les programmes en évaluant des conditions.

Opérateur	Objectif
<	inférieur
<=	inférieur ou égale
>	supérieur
>=	supérieur ou égale
==	égale
!=	différent

1.14. Les opérateurs relationnels

```
int a = 20, b = 10;
```

Opérateur	Exemple	Résultat
<	<code>a < b</code>	0
<=	<code>a <= b</code>	0
>	<code>a > b</code>	1
>=	<code>a >= b</code>	1
==	<code>a == b</code>	0
!=	<code>a != b</code>	1

Remarque: Une valeur différente de zéro est considérée comme VRAIE et la valeur zéro est considérée comme FAUSSE.

1.15. Les opérateurs logiques

Les opérateurs logiques en C, tels que "&&" (ET logique), "||" (OU logique) et "!" (NON logique), sont utilisés pour combiner des expressions booléennes et effectuer des opérations de logique booléenne, ce qui permet de prendre des décisions conditionnelles complexes en évaluant plusieurs conditions simultanément dans les programmes.

```
int a = 20, b = 0;
```

Opérateur	Objectif	Exemple	Résultat
!	Négation	!a	0
&&	Et	a && b	0
	Ou	a b	1

Remarque: Une valeur différente de zéro est considérée comme VRAIE et la valeur zéro est considérée comme FAUSSE.

1.16. Les opérateurs d'incrémentation

Les opérateurs d'incrémentation en C, "++" (incrément) et "--" (décrément), sont utilisés pour augmenter ou diminuer la valeur d'une variable numérique de 1, ce qui facilite les itérations et les mises à jour de variables dans les boucles et les calculs.

```
int a = 20, b = 0;
```

Opérateur	Exemple	Résultat
a++	b = a++	a = 21, b = 20
++a	b = ++a	a = 21, b = 21
a--	b = a--	a = 19, b = 20
--a	b = --a	a = 19, b = 19

1.17. Les opérateurs de manipulation de bits

Les opérateurs de manipulation de bits en C, tels que "&" (ET bit à bit), "|" (OU bit à bit), "^" (OU exclusif bit à bit) et "~" (NON bit à bit), permettent de réaliser des opérations de manipulation au niveau des bits au sein des données binaires, ce qui est utile pour effectuer des opérations de masquage, de décalage et de gestion de bits individuels dans les programmes.

Bit 1	Bit 2	&		^ (XOR)
0	0	0	0	0
0	1	0	1	1
1	0	0	1	1
1	1	1	1	0

Remarque: Les valeurs ci-dessus (0 et 1) sont des bits binaires et non des valeurs entières.

1.17. Les opérateurs de manipulation de bits

```
int a = 0b01000100;
```

Opérateur	Objectif	Exemple	Résultat
~	Négation	~a	0xfffffbb
&	ET	a & 0x4	0x4
	OR	a 0x2	0x46
^	XOR	a ^ 0x4	0x40
<<	décalage à gauche	a << 1	0x88
>>	décalage à droite	a >> 1	0x22

1.18. Les opérateurs d'affectation

Les opérateurs d'affectation de bits en C, tels que "&=", "|=", "^=", "<<=", et ">>=", sont utilisés pour combiner des opérations de manipulation de bits avec une opération d'affectation. Ils permettent de modifier les valeurs d'une variable en fonction du résultat d'une opération de manipulation de bits, ce qui simplifie la gestion des données au niveau des bits. **Remarque:** $a \text{ op } b \text{ ::- } a = a \text{ op } b$

Opérateur	Objectif	Exemple
=	equal	$a = b$
+=	addition	$a += b$
-=	substraction	$a -= b$
*=	multiplication	$a *= b$
/=	division	$a /= b$
%=	modulo	$a \% = b$

1.18. Les opérateurs d'affectation

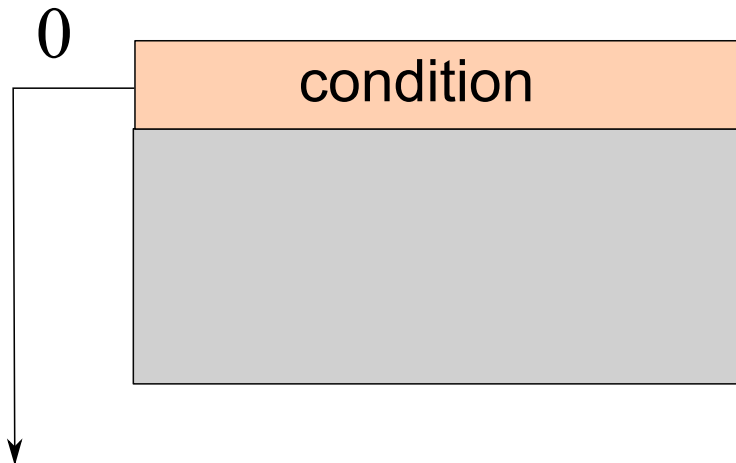
Opérateur	Objectif	Exemple
<code>&=</code>	ET	<code>a &= b</code>
<code> =</code>	OU	<code>a = b</code>
<code>^=</code>	XOR	<code>a ^= b</code>
<code><<=</code>	décalage à gauche	<code>a <<= b</code>
<code>>>=</code>	décalage à droite	<code>a >>= b</code>

Remarque: `a op = b` ::- `a = a op b`

1.19. Structures de contrôle: les branchements conditionnels

if

```
if (condition) {  
    ...  
}
```



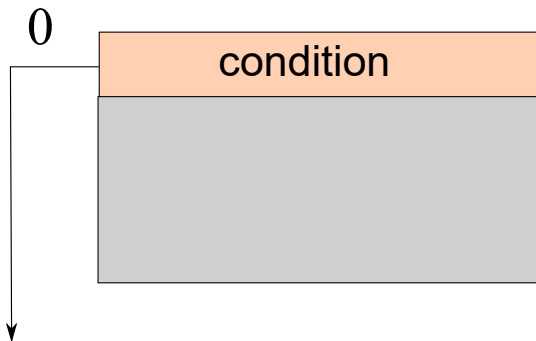
- **condition** est une expression booléenne. Elle doit retourner soit true (vraie) soit false (fausse). Si la condition est vraie, le code à l'intérieur du bloc if est exécuté. Si la condition est fausse, le code à l'intérieur du bloc if est ignoré.
- Le bloc de code à l'intérieur des accolades {} est le code qui sera exécuté si la condition est vraie. Vous pouvez y mettre n'importe quel nombre d'instructions.
- **Remarque:** C n'a pas de type booléen. Une valeur différente de zéro est considérée comme VRAIE et la valeur zéro est considérée comme FAUSSE.

1.19. Structures de contrôle: les branchements conditionnels

if

```
int a = 20, b = 0;  
if (a > b) {  
    printf("a est supérieur à b");  
}
```

L'objectif est de vérifier si la valeur de `a` est supérieure à celle de `b` et d'afficher un message si la condition est vraie.



1.19. Structures de contrôle: les branchements conditionnels

if

Les instructions `if...else if...else` permettent d'exécuter un bloc de code en fonction de plusieurs conditions alternatives.

```
if (condition1) {  
    ...  
} else if (condition2) {  
    ...  
} else {  
    ...  
}
```

- Si `condition1` est vraie, le code du premier bloc est exécuté.
- Si `condition1` est fausse et `condition2` est vraie, le code du deuxième bloc (`else if`) est exécuté.
- Si aucune des conditions précédentes n'est vraie, le code du bloc `else` est exécuté, agissant comme une clause par défaut.

1.19. Structures de contrôle: les branchements conditionnels

if

Les instructions `if...else if...else` permettent d'exécuter un bloc de code en fonction de plusieurs conditions alternatives.

```
if (condition1) {  
    ...  
} else if (condition2) {  
    ...  
} else {  
    ...  
}
```

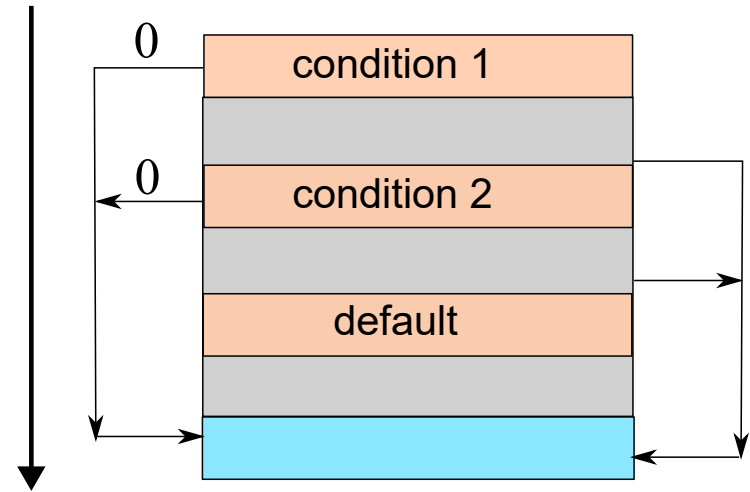
Remarque 1: Toutes les nombres non nulles sont ont considérées comme vrai ('TRUE')

Remarque 2: `else` est optionnel

1.19. Structures de contrôle: les branchements conditionnels

if

```
int a = 20, b = 0;
if (a > b) {
    printf("a est supérieur
à b");
} else if (a < b) {
    printf("a est inférieur
à b");
} else {
    printf("a égale b");
}
```



1.20. Structures de contrôle: les branchements conditionnels (choix)

switch

`switch` est utilisée pour sélectionner une action à exécuter en fonction de la valeur de l'expression fournie.

```
switch (expression) {  
    case valeur1 : instructions1  
    case valeur2 : instructions2  
    ...  
    default : instructionsn  
}
```

1.20. Structures de contrôle: les branchements conditionnels (choix)

switch

```
switch (expression) {  
    case valeur1 : instructions1  
    case valeur2 : instructions2  
    ...  
    default : instructionsn  
}
```

- **Remarque:** : Il est important d'utiliser `break` à la fin de chaque `case` pour éviter une "chute à travers" involontaire, où le programme exécuterait les instructions de plusieurs `case` jusqu'à ce qu'il atteigne un `break`.
- **Remarque:** `expression` doit être soit: `char`, `short`, `int` ou `long`.

1.20. Structures de contrôle: les branchements conditionnels (choix)

switch

```
int a = 20;  
switch (a) {  
    case 10 : instructions1  
        break;  
    case 20 : instructions2  
    case 30 : instructions3  
        break;  
    ...  
    default : instructionsn  
}
```

Remarque: Le code exécute `instructions2` et continue à exécuter `instructions3` car il n'y a pas de `break` après `instructions3`.

Que sera-t-il affiché à l'écran suite à l'exécution de ce programme?

```
if (1) {  
    printf("Hi");  
} else {  
    printf("Bonjour");  
}
```

1.21. Structures de contrôle: les boucles

for

```
for(initialisation;condition;actualisation){  
    ...  
}
```

L'objectif de la boucle **for** en programmation est de répéter un bloc de code un certain nombre de fois en suivant les étapes suivantes

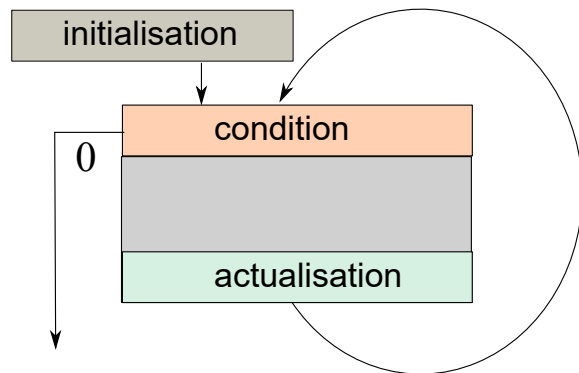
- **Initialisation** : Cette étape est effectuée une seule fois au début de la boucle. Elle est utilisée pour définir ou initialiser une variable de contrôle.
- **Condition** : À chaque itération de la boucle, la **condition** est évaluée. Si la **condition** est vraie (true), le bloc de code à l'intérieur de la boucle est exécuté. Si la **condition** est fausse (false), la boucle se termine.
- **Actualisation** : Après chaque exécution du bloc de code, l'étape d'**actualisation** est effectuée. Elle est généralement utilisée pour modifier la variable de contrôle, ce qui peut éventuellement changer la condition de la boucle.

1.21. Structures de contrôle: les boucles

for

Ces étapes permettent de créer des boucles flexibles qui peuvent être utilisées pour parcourir des éléments d'une liste, effectuer des calculs répétitifs, ou exécuter des actions itératives. La boucle **for** est couramment utilisée pour simplifier la gestion des itérations dans un programme.

```
for(initialisation;condition;actualisation){  
    ...  
}
```

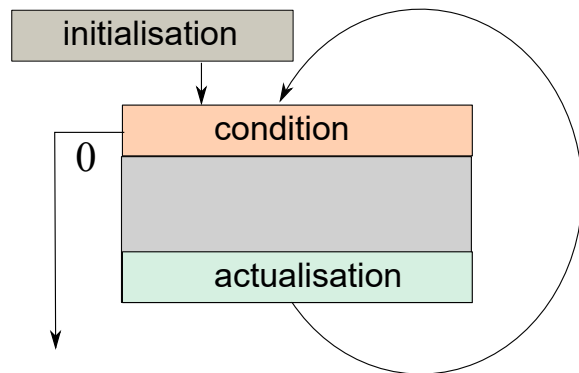


1.21. Structures de contrôle: les boucles

for

```
int a = 0;  
for( a = 0; a < 10; a++){  
    ...  
}
```

L'objectif de ce code est d'initialiser la variable `a` à 0, puis de répéter un bloc de code tant que `a` est inférieur à 10, en augmentant la valeur de `a` à chaque itération.



1.21. Structures de contrôle: les boucles

for

```
int a = 0;  
for(; a < 10; ){  
    ...  
}
```

L'objectif de ce code est de répéter un bloc de code tant que la variable `a` est inférieure à 10, en utilisant une initialisation préalable de `a` à 0 et en omettant l'actualisation de `a` à l'intérieur de la boucle.

Remarque: Une ou toutes les instructions d'initialisation, de condition ou d'actualisation peuvent être manquant.

1.21. Structures de contrôle: les boucles

for

```
int a = 0;  
for( a = 0; a < 10; a++){  
    ...  
    a += 2 ;  
    ...  
}
```

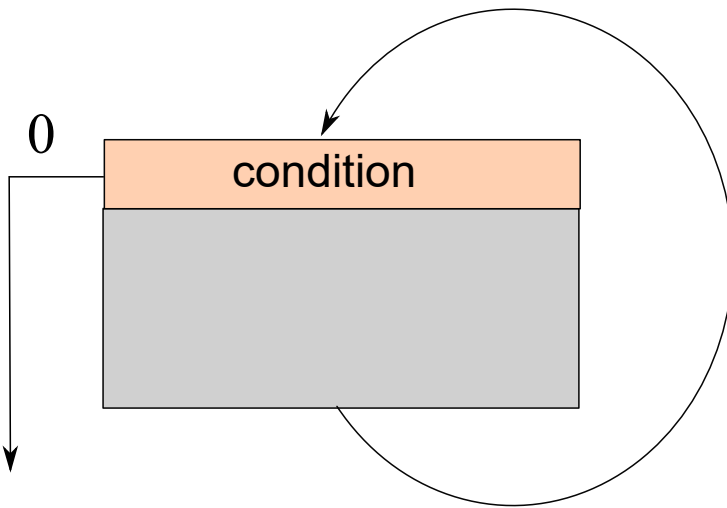
L'objectif de ce code est d'initialiser la variable `a` à 0, puis de répéter un bloc de code tant que `a` est inférieur à 10, en augmentant la valeur de `a` de 2 à chaque itération à l'intérieur de la boucle, tout en incrémentant `a` de 1 avec l'opérateur `a++` à chaque fin d'itération.

1.21. Structures de contrôle: les boucles

while

La boucle **while** maintient l'exécution d'un bloc de code tant que la **condition** entre les parenthèses reste vraie.

```
while(condition){  
    ...  
}
```

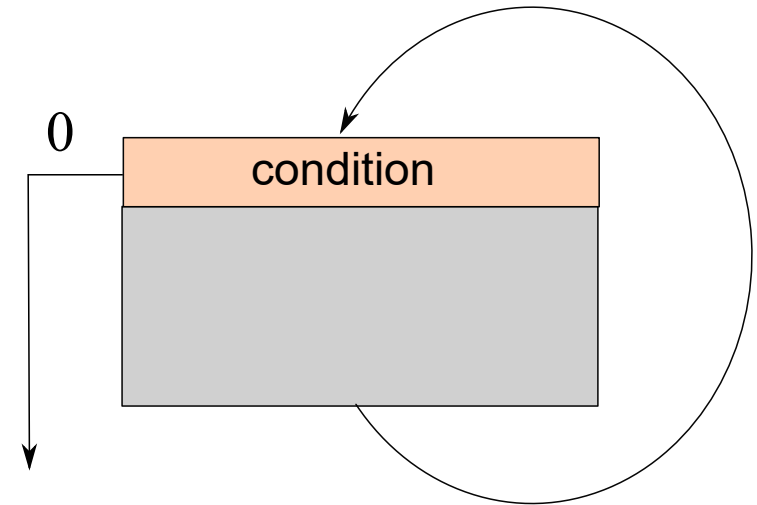


1.21. Structures de contrôle: les boucles

while

```
int a = 20;  
while(a > 0){  
    ...  
    a--;  
    ...  
}
```

Ce code maintient la répétition d'un bloc de code tant que la valeur de la variable `a` est supérieure à 0, en la décrémentant à chaque itération.



while

```
int a = 0;  
while(a < 20){  
    ...  
    a++;  
    ...  
}
```

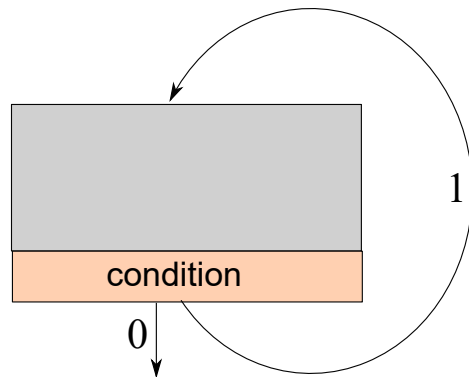
Ce code vise à répéter un bloc de code tant que la valeur de la variable `a` est inférieure à 20, en l'incrémentant à chaque itération.

1.21. Structures de contrôle: les boucles

do..while

La boucle `do..while` garantit l'exécution d'un bloc de code au moins une fois avant de répéter son exécution tant que la `condition` spécifiée reste vraie.

```
do{  
    ...  
} while(condition);
```

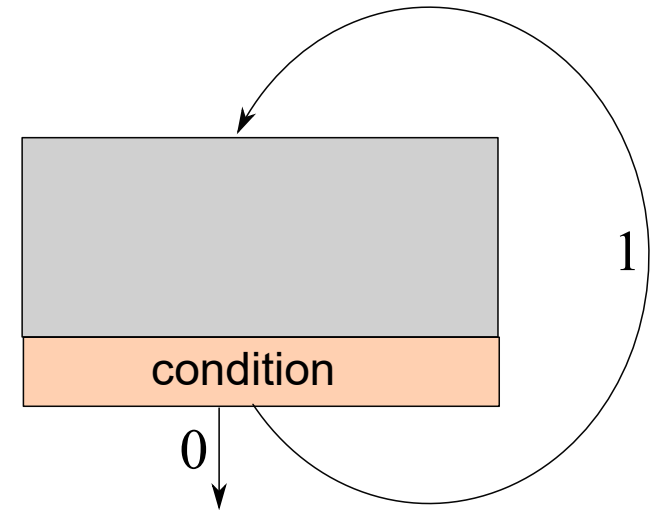


1.21. Structures de contrôle: les boucles

do..while

```
int a = 20;  
do{  
    ...  
    a --;  
    ...  
} while(a > 0);
```

Ce code exécute un bloc de code au moins une fois, puis le répète tant que la valeur de la variable `a` est supérieure à 0, en décrémentant `a` à chaque itération.



do..while

```
int a = 0;  
do{  
    ...  
    a ++;  
    ...  
} while(a < 20);
```

Ce code garantit l'exécution d'un bloc de code au moins une fois, suivi de répétitions tant que la valeur de la variable `a` est inférieure à 20, en l'incrémentant à chaque itération.

1.22. Structures de contrôle: les sauts (branchements)

break

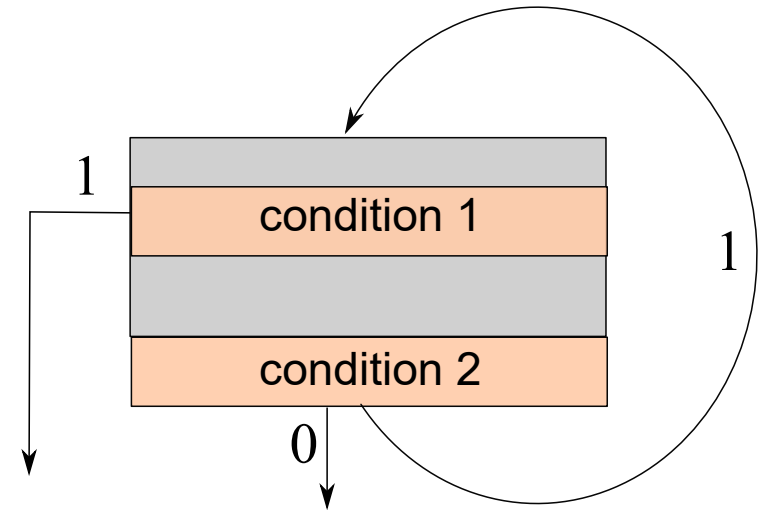
L'instruction `break` est utilisée pour interrompre prématurément l'exécution d'une boucle (par exemple, `for` ou `while`) ou d'un bloc de commutation `switch`. Lorsque `break` est rencontré, le programme sort immédiatement de la structure de contrôle dans laquelle il se trouve, sans attendre la fin normale de la condition ou de la boucle. Cela permet de quitter la structure dès qu'une condition souhaitée est atteinte ou lorsqu'une sortie anticipée est nécessaire.

```
do{
    ...
    if (condition 1) {
        ...
        break;
    }
    ...
} while(condition 2);
```

1.22. Structures de contrôle: les sauts (branchements)

break

```
do{  
    ...  
    if (condition 1) {  
        ...  
        break;  
    }  
    ...  
} while(condition 2);
```



Ce code vise à exécuter un bloc de code au moins une fois, puis à répéter son exécution tant que **condition 2** est vraie, tout en utilisant **break** pour sortir de la boucle si **condition 1** est satisfaite.

1.22. Structures de contrôle: les sauts (branchements)

continue

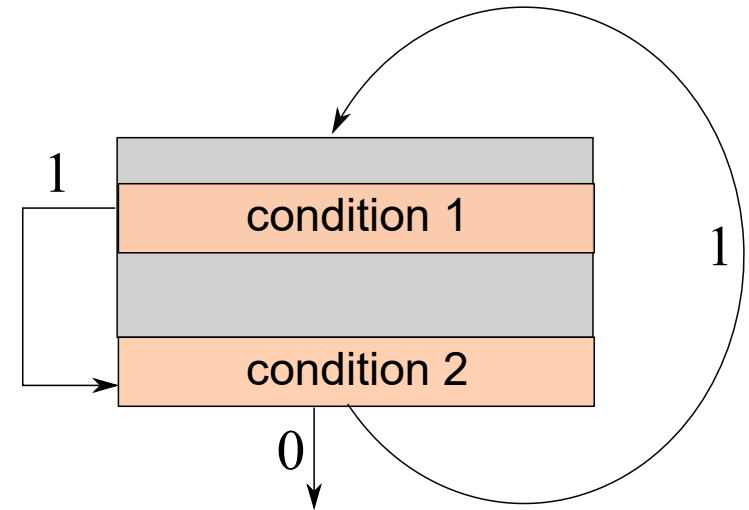
L'instruction `continue` est utilisée pour passer immédiatement à l'itération suivante d'une boucle (par exemple, `for` ou `while`), en ignorant le reste du code à l'intérieur de la boucle pour cette itération particulière. Elle permet de sauter l'exécution des instructions restantes dans la boucle et de passer à la prochaine itération, ce qui est utile pour gérer des cas spécifiques sans quitter complètement la boucle.

```
do{  
    ...  
    if (condition 1) {  
        ...  
        continue;  
    }  
    ...  
} while(condition 2);
```

1.22. Structures de contrôle: les sauts (branchements)

continue

```
do{  
    ...  
    if (condition 1) {  
        ...  
        continue;  
    }  
    ...  
} while(condition 2);
```

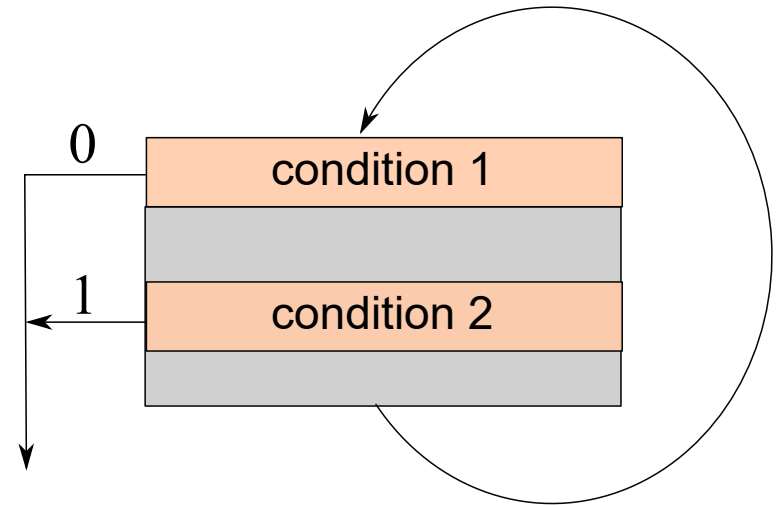


Ce code a pour objectif d'exécuter un bloc de code au moins une fois, puis de le répéter tant que **condition 2** est vraie, en utilisant **continue** pour sauter le reste du code dans une itération si **condition 1** est satisfaite.

1.22. Structures de contrôle: les sauts (branchements)

break

```
while(condition 1){  
    ...  
    if (condition 2) {  
        ...  
        break;  
    }  
    ...  
};
```

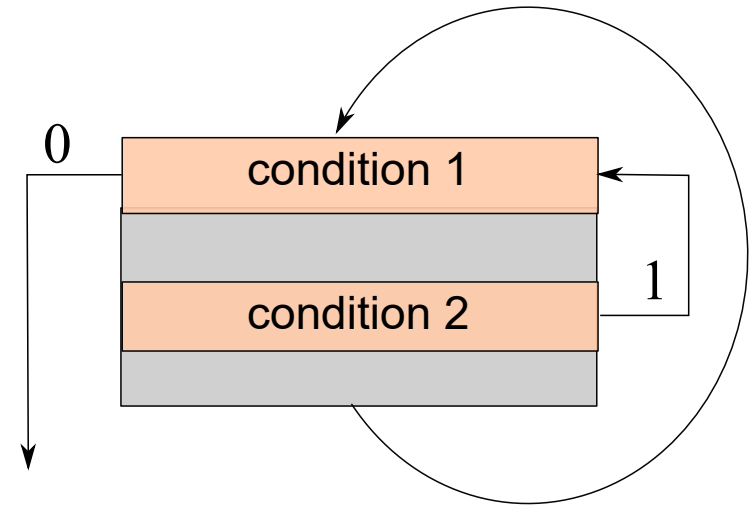


Dans ce code, l'objectif est d'exécuter un bloc de code tant que **condition 1** est vraie, en utilisant **break** pour sortir de la boucle si **condition 2** est satisfaite à l'intérieur de cette boucle.

1.22. Structures de contrôle: les sauts (branchements)

continue

```
while(condition 1){  
    ...  
    if (condition 2) {  
        ...  
        continue;  
    }  
    ...  
};
```



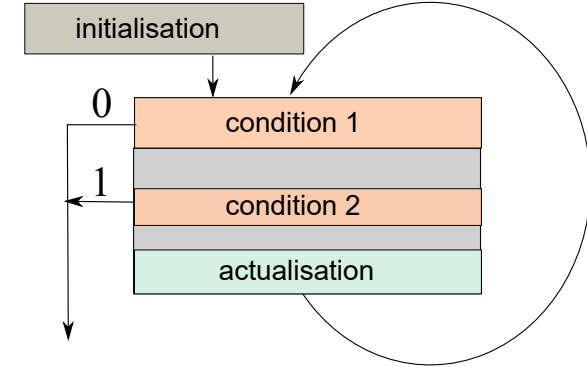
Dans ce code, l'objectif est d'exécuter un bloc de code tant que **condition 1** est vraie, en utilisant **continue** pour passer à l'itération suivante de la boucle si **condition 2** est satisfaite à l'intérieur de cette boucle.

1.22. Structures de contrôle: les sauts (branchements)

break

```
for(initialisation;condition 1;actualisation){  
    ...  
    if (condition 2) {  
        ...  
        break;  
    }  
    ...  
};
```

L'objectif de ce code avec la boucle "for" est d'initialiser une variable, puis de répéter un bloc de code tant que **condition 1** est vraie, en utilisant **break** pour sortir de la boucle si **condition 2** est satisfaite à l'intérieur de celle-ci.

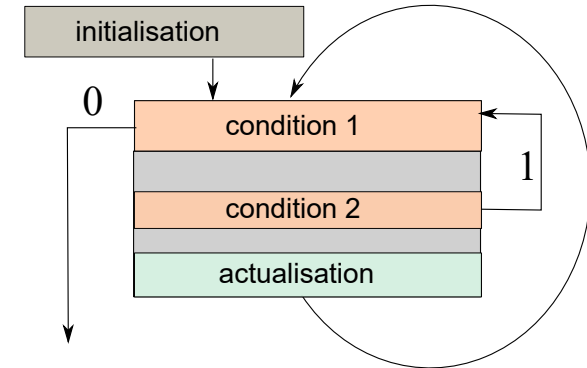


1.22. Structures de contrôle: les sauts (branchements)

continue

```
for(initialisation;condition 1;actualisation){  
    ...  
    if (condition 2) {  
        ...  
        continue;  
    }  
    ...  
};
```

L'objectif de ce code avec la boucle "for" est d'initialiser une variable, puis de répéter un bloc de code tant que **condition 1** est vraie, en utilisant **continue** pour sauter le reste du code dans une itération si **condition 2** est satisfaite à l'intérieur de la boucle.



Écrivez un programme qui affiche 0..20 en ordre croissant et décroissant utilisant for, while et do..while.

Écrivez un boucle infini en utilisant for, while, do..while.

1.23. Les tableaux

Tableau

Les tableaux sont des structures de données permettant de stocker **un ensemble de valeurs du même type** sous un même nom, organisées de manière **séquentielle**.

B	o	n	j	o	u	r
---	---	---	---	---	---	---

12	1	5
----	---	---

1.23. Les tableaux

Tableaux de tableaux

Les **tableaux à deux indices** sont des structures de données qui permettent de stocker des données dans une matrice rectangulaire.

Les tableaux à deux indices sont utiles pour stocker des **données tabulaires**, des images, des matrices, etc.

12	1	5
3	8	9

1.23. Les tableaux

Tableaux de tableaux

Les **tableaux à plusieurs indices** sont des structures de données permettant de stocker des données dans des dimensions multiples.

Les tableaux à plusieurs indices sont utilisés pour stocker des **données tridimensionnelles**, telles que des voxels dans une image 3D, des matrices 3D, des jeux de données multidimensionnels, etc.



1.23. Les tableaux

Déclaration: Tableaux de caractères

Pour déclarer un tableau, spécifiez le **type** des éléments qu'il contiendra, suivi du **nom du tableau** et de sa **taille** entre crochets.

par exemple

```
int tableau[5];
```

déclare un tableau d'entiers de taille 5.

```
char nom[20];
```

déclare un tableau de caractères (ou une chaîne de caractères) de taille 20.

Remarque: Langage C n'a pas un type nommé 'string'.

1.23. Tableaux

Déclaration: taille fixe

Les tableaux ont une taille fixe à la déclaration, qui ne peut pas être modifiée pendant l'exécution.

Exemple :

```
int itableau[20];  
float ftableau[20];  
double dtableau[20];
```

Objectif :

- Trois tableaux ont été déclarés : `itableau`, `ftableau`, et `dtableau`.
Chaque tableau peut stocker jusqu'à 20 éléments.
- `itableau` est un tableau d'entiers.
- `ftableau` est un tableau de nombres à virgule flottante (float).
- `dtableau` est un tableau de nombres à virgule flottante double précision (double).

1.23. Tableaux

Initialisation: tableau à une dimension

Exemple :

```
int i;
int tableau[20];
for ( i = 0; i < 20; i++) {
    tableau[i] = i;
}
```

- L'objectif est d'initialiser un tableau d'entiers nommé `tableau` de taille 20 en utilisant une boucle `for`.
- La boucle `for` est utilisée pour parcourir les indices de 0 à 19 (20 éléments au total).
- À chaque itération de la boucle, la valeur de `i` est assignée à l'élément correspondant du tableau `tableau`.
- Cela signifie que `tableau[0]` sera égal à 0, `tableau[1]` sera égal à 1, `tableau[2]` sera égal à 2, et ainsi de suite jusqu'à `tableau[19]` qui sera égal à 19.
- En fin de compte, le tableau `tableau` sera initialisé avec une séquence de nombres de 0 à 19.

1.23. Les tableaux

Initialisation: tableau à une dimension

- L'initialisation de tableaux en C permet de définir les valeurs initiales des éléments du tableau au moment de la déclaration.
- Les valeurs initiales sont placées entre des accolades {} et séparées par des virgules.
- Les crochets [] indiquent que la taille du tableau sera déduite automatiquement du nombre d'éléments fournis.
- L'initialisation permet de pré-remplir un tableau avec des valeurs spécifiques, ce qui peut être utile dans de nombreuses situations en programmation.

Exemple :

```
int prix[5] = { 11, 12, 13, 14, 15 };  
int chambres[] = { 301, 302, 303 };  
char message[] = "Bonjour Le Monde!!";
```

1.23. Les tableaux

Initialisation: tableau à une dimension

Exemple :

```
int prix[5] = { 11, 12, 13, 14, 15 };  
sizeof (prix) // 5 * sizeof(int)
```

1.23. Les tableaux

Initialisation: tableau à une dimension

Exemple :

```
int chambres[] = { 301, 302, 303 };
```

- déclare un tableau d'entiers initialisé avec trois valeurs.
- Lorsque vous utilisez `sizeof (chambres)`, cela renverra la taille totale en octets de ce tableau, c'est-à-dire la taille de chaque élément (un entier) multipliée par le nombre d'éléments (3).
- Si un entier est généralement stocké sur 4 octets (32 bits) dans votre système, `sizeof (chambres)` renverra 12 (3 entiers * 4 octets par entier).

```
sizeof (chambres) // 3 * sizeof(int)
```

1.23. Les tableaux

Initialisation: tableau à une dimension

Exemple :

```
char message[] = "Bonjour Le Monde!!";
```

- déclare un tableau de caractères (chaîne de caractères) initialisé avec la phrase "Bonjour Le Monde!!".
- Lorsque vous utilisez `sizeof (message)`, cela renverra la taille totale en octets de la chaîne de caractères, y compris le caractère nul de fin de chaîne ('\0'), qui est automatiquement ajouté à la fin d'une chaîne en C.
- Si chaque caractère est généralement stocké sur 1 octet (8 bits) dans votre système, `sizeof (message)` renverra la taille totale de la chaîne, y compris le caractère nul, soit la longueur de la phrase plus 1 pour le caractère nul.

```
sizeof (message) // (18 + 1 (NULL)) * sizeof(char)
```

1.23. Les tableaux

Tableaux à deux indices

```
int prix[2][2] = {  
    {11, 12},  
    {13, 14}  
};  
  
sizeof (prix) // 2 * 2 * sizeof(int)
```

- `int prix[2][2]` déclare un tableau à deux dimensions nommé "prix" avec une taille de 2 lignes et 2 colonnes.
- Les valeurs entre les accolades `{}` sont les données du tableau. Dans ce cas, vous avez deux lignes et deux colonnes, avec les valeurs 11, 12, 13 et 14.
- Pour accéder à un élément spécifique du tableau, vous utilisez des indices, par exemple `prix[0][0]` pour accéder à 11, `prix[1][1]` pour accéder à 14, etc.

1.23. Les tableaux

Initialisation: Tableaux à deux indices

```
int chambres[][2] = {  
    {201, 202},  
    {301, 302}  
};  
sizeof (chambres) // 2 * 2  
* sizeof(int)
```

Remarque: Nous n'avons pas précisé le nombre de lignes pour `chambres`.

- `int chambres[][2]` déclare un tableau à deux dimensions nommé `chambres` sans spécifier la taille exacte. Cela signifie que le compilateur détermine automatiquement le nombre de lignes en fonction du nombre de paires d'accolades internes. Dans cet exemple, il y a deux lignes et deux colonnes.
- Les valeurs entre les accolades `{}` représentent les données du tableau. Vous avez deux lignes et deux colonnes, avec les valeurs 201, 202, 301 et 302.
- Pour accéder à un élément spécifique du tableau, vous utilisez des indices, par exemple `chambres[0][0]` pour accéder à 201, `chambres[1][1]` pour accéder à 302, etc.

1.23. Les tableaux

Initialisation: Tableaux à deux indices

```
char message[2][11] = {"Bonjour", "Le Monde!!"};  
sizeof (message) // 2 * 11 * sizeof(char)
```

- `char message[2][11]` déclare un tableau à deux dimensions nommé `message` avec deux lignes et 11 colonnes. Cela signifie que le tableau peut stocker deux chaînes de caractères, chacune pouvant avoir jusqu'à 10 caractères, plus un caractère pour le terminateur nul (`'\0'`).
- Les valeurs entre les guillemets `{"Bonjour", "Le Monde!!"}` sont les données du tableau. Vous avez deux chaînes de caractères, `"Bonjour"` dans la première ligne et `"Le Monde!!"` dans la deuxième ligne.
- Chaque élément du tableau, comme `message[0]`, est une chaîne de caractères (tableau de caractères) pouvant être manipulée comme n'importe quelle autre chaîne de caractères.
- Pour accéder à un caractère spécifique dans une chaîne, vous utilisez des indices supplémentaires. Par exemple, `message[0][0]` accède au premier caractère de la première chaîne, c'est-à-dire `'B'`.

1. Programmation en C: Références

Références

- Langage C, Claude Delannoy
- [https://fr.wikipedia.org/wiki/C_\(langage\)](https://fr.wikipedia.org/wiki/C_(langage))
- https://en.wikipedia.org/wiki/C_data_types

Crédits d'images

- [Wikimedia Commons](#)